

INSTITUTO FEDERAL DE SANTA CATARINA

Disciplina: Engenharia de Software I

Prof. Rogério Pereira Junior

SISTEMA DE CAFETERIA

Projeto Final — Relatório Técnico

Estudantes:

Mônica Cancellier | Nicolas Franzosi | Victor Eduardo

Junho de 2026

2.1 Processo de Software

Para o desenvolvimento do Sistema de Cafeteria, o grupo adotou o processo de software incremental. Esse modelo possibilita desenvolver o sistema em etapas, entregando funcionalidades de forma gradual e realizando ajustes durante o projeto.

Inicialmente foram levantados os requisitos do sistema, identificando as principais necessidades: cadastro de usuários, cadastro de produtos, gerenciamento de pedidos, controle de clientes e emissão de relatórios. Após essa etapa, o desenvolvimento foi dividido em incrementos, nos quais cada funcionalidade foi implementada e validada separadamente.

Justificativa da Escolha

O modelo incremental foi escolhido pelas seguintes razões:

- Permite validar cada funcionalidade antes de avançar para a próxima, reduzindo retrabalho.
- Facilita a distribuição de tarefas entre os membros da equipe de forma independente.
- Possibilita maior coesão, pois cada classe foi criada com função bem definida antes da integração.
- Reduz riscos: erros são identificados cedo, em escopo menor.

Organização das Atividades

As responsabilidades foram distribuídas conforme as competências de cada integrante:

Atividade	Responsável(eis)
Levantamento de requisitos	Todos os integrantes
Elaboração dos diagramas UML	Todos os integrantes
Implementação: Cadastro de Produtos	Nicolas Franzosi
Implementação: Cadastro de Clientes	Mônica Cancellier
Implementação: Gerenciamento de Pedidos	Victor Eduardo
Testes unitários e uso de mocks	Todos os integrantes
Elaboração do relatório	Todos os integrantes

Incrementos do Projeto

Incremento	Funcionalidade	Status
1	Cadastro de Produtos (CRUD)	Concluído
2	Cadastro de Clientes	Concluído
3	Registro e Gerenciamento de Pedidos	Concluído

Incremento	Funcionalidade	Status
4	Cálculo automático do valor do pedido	Concluído
5	Atualização de status do pedido	Concluído

2.2 Requisitos de Software

Requisitos Funcionais

ID	Requisito
RF01	O sistema deve permitir o cadastro, edição e exclusão de produtos.
RF02	O sistema deve permitir o cadastro de clientes.
RF03	O sistema deve permitir o registro de pedidos realizados pelos clientes.
RF04	O sistema deve permitir consultar os produtos disponíveis.
RF05	O sistema deve calcular automaticamente o valor total do pedido.
RF06	O sistema deve permitir atualizar o status do pedido.

Requisitos Não Funcionais

ID	Requisito
RNF01	O sistema deve possuir interface simples e intuitiva.
RNF02	O sistema deve apresentar tempo de resposta adequado para consultas e cadastros.
RNF03	Os dados dos clientes, produtos e pedidos devem ser armazenados de forma segura.
RNF04	O sistema deve possuir código organizado e de fácil entendimento.
RNF05	O sistema deve garantir a integridade e a disponibilidade das informações armazenadas.

Histórias de Usuário

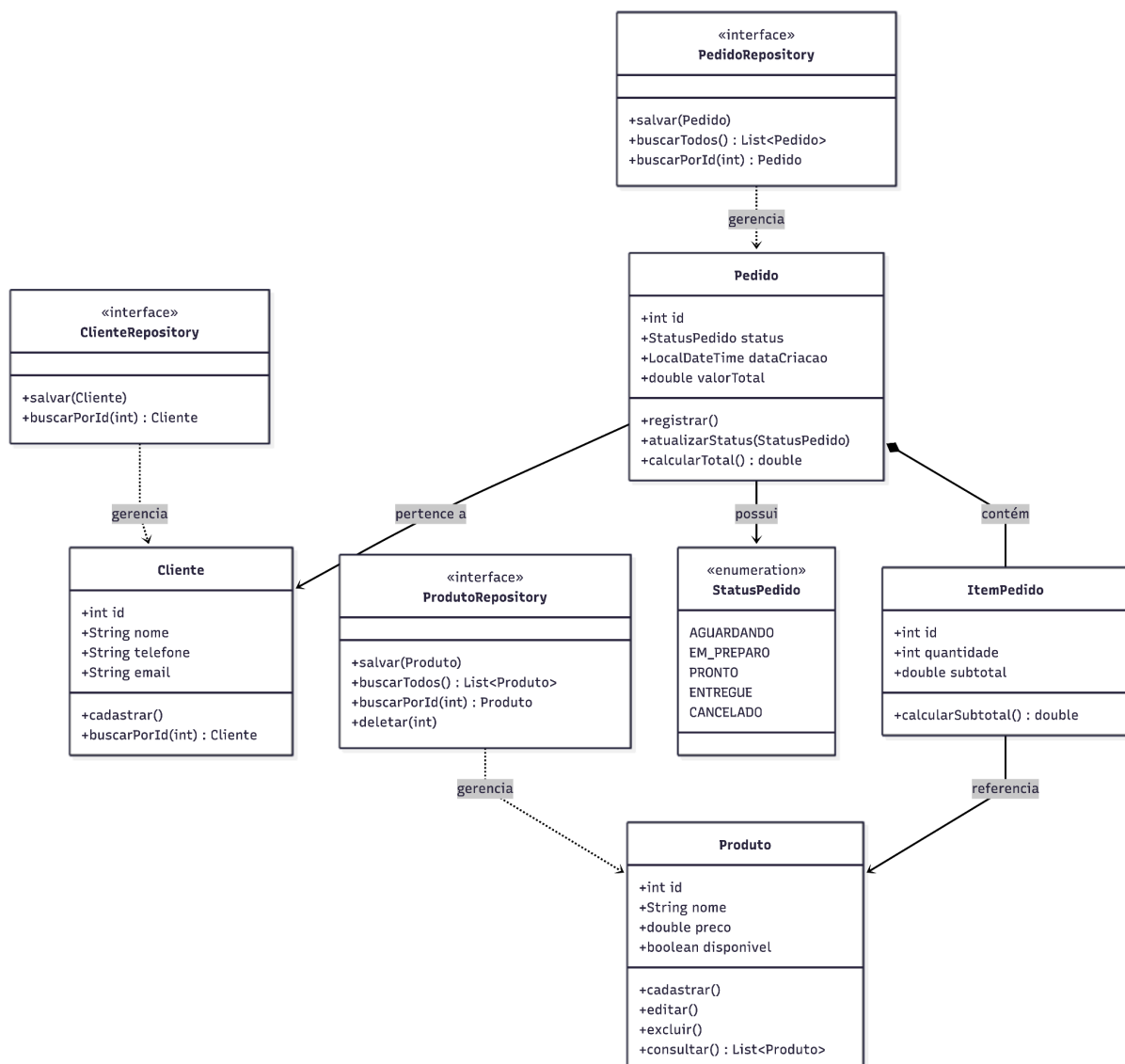
ID	Papel	Desejo	Para que
HU01	Atendente	Cadastrar clientes	Registrar pedidos de forma organizada
HU02	Atendente	Registrar pedidos	Agilizar o atendimento
HU03	Atendente	Visualizar informações organizadas	Realizar atendimentos com rapidez e reduzir erros
HU04	Atendente	Cadastrar produtos	Manter o cardápio atualizado
HU05	Cliente	Visualizar os produtos	Escolher a compra
HU06	Gerente	Visualizar pedidos realizados	Controlar o funcionamento do estabelecimento
HU07	Cliente	Visualizar resumo do pedido	Conferir produtos escolhidos e valor total
HU08	Atendente	Que cada produto tenha nome e preço	Facilitar o registro dos pedidos

2.3 Modelo de Software

Esta seção apresenta os diagramas UML obrigatórios: diagrama de classes, diagrama de casos de uso e um diagrama dinâmico de sequência. Os códigos Mermaid abaixo permitem a renderização de cada diagrama em ferramentas como Mermaid Live Editor (<https://mermaid.live>).

Diagrama de Classes

O diagrama de classes representa as entidades do sistema, seus atributos, métodos e os relacionamentos entre elas. O sistema é dividido em: entidades de domínio (Produto, Cliente, Pedido, ItemPedido), enumeração de status e interfaces de repositório.



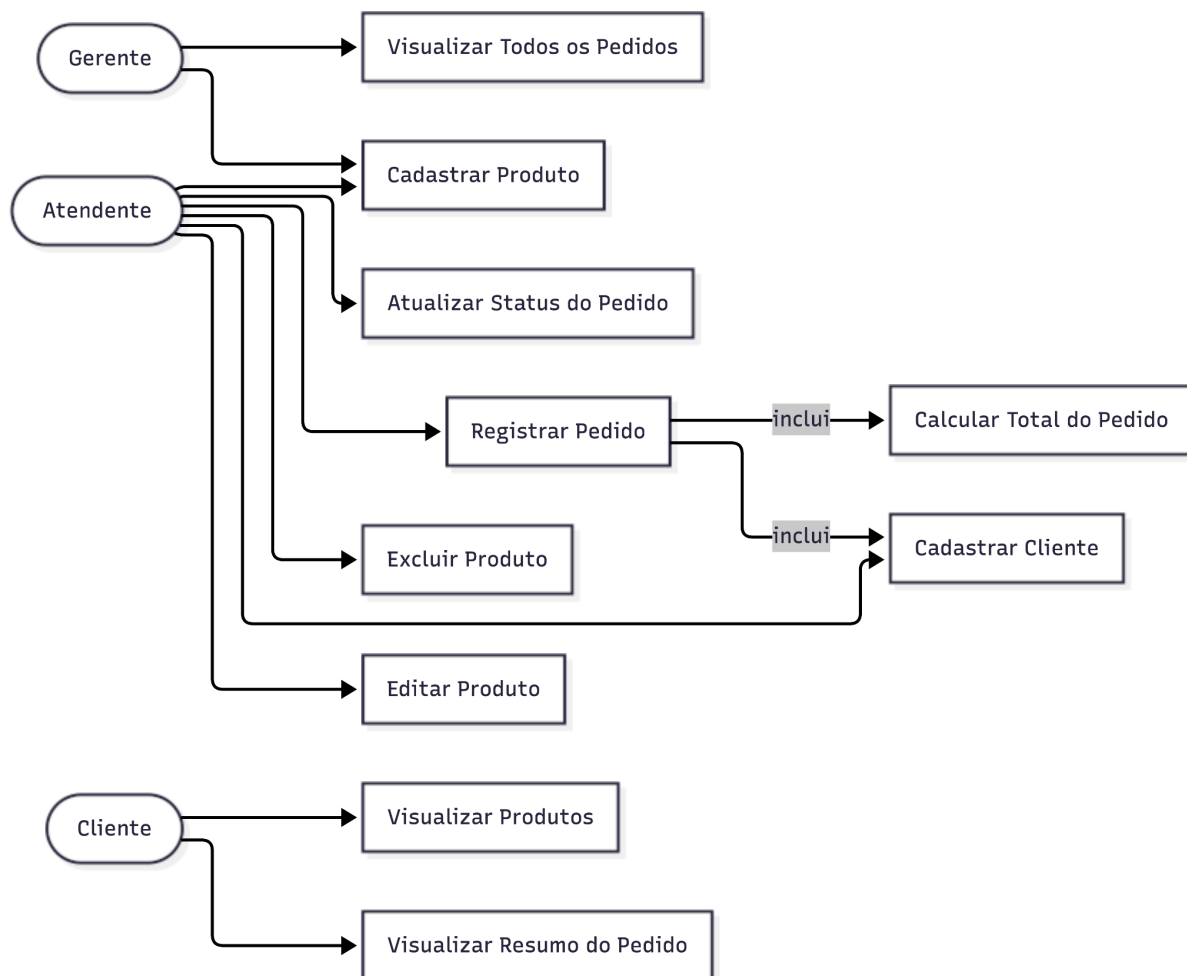
Descrição dos relacionamentos:

- Pedido pertence a um Cliente (associação).

- Pedido contém um ou mais ItemPedido (composição).
- ItemPedido referencia um Produto (associação).
- Pedido possui um StatusPedido (enumeração).
- As interfaces Repository realizam a abstração de persistência para cada entidade (Princípio da Inversão de Dependência - SOLID).

Diagrama de Casos de Uso

O diagrama de casos de uso descreve as interações entre os atores e as funcionalidades do sistema. Os atores identificados são: Atendente, Cliente e Gerente.

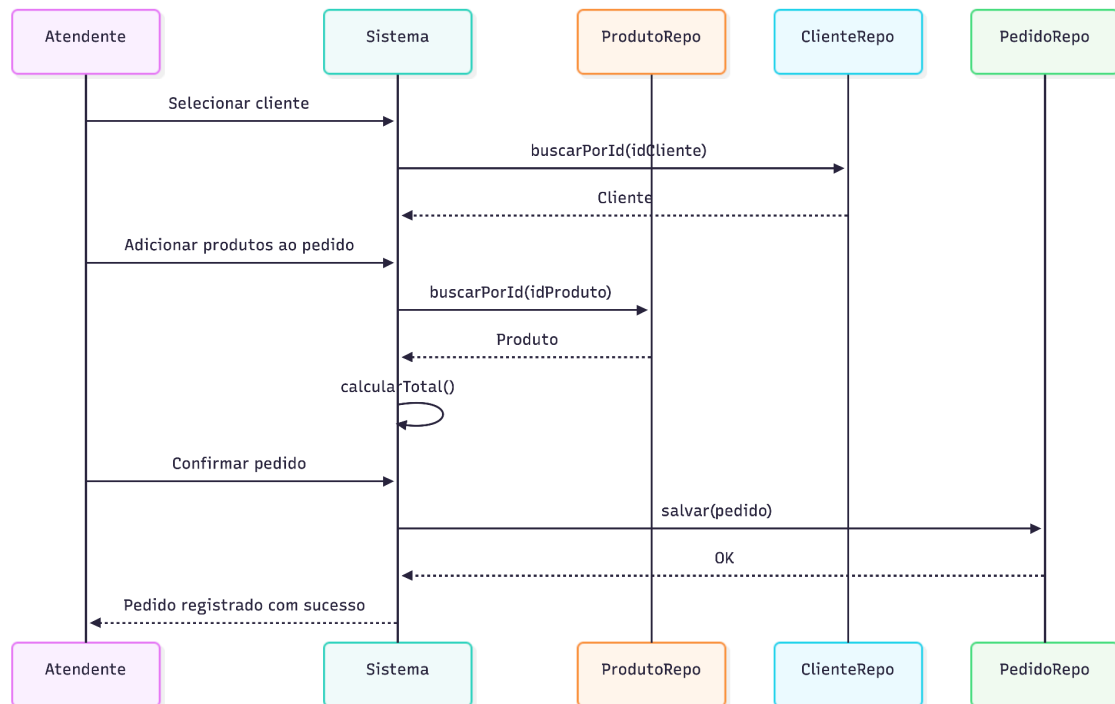


Descrição dos atores:

- Atendente: responsável pelo gerenciamento operacional (produtos, clientes, pedidos).
- Cliente: interage com o sistema para visualizar produtos e acompanhar seu pedido.
- Gerente: possui visão ampla dos pedidos e acesso ao gerenciamento de produtos.

Diagrama de Sequência — Registrar Pedido

O diagrama de sequência ilustra o fluxo dinâmico da funcionalidade central do sistema: o registro de um pedido pelo atendente, desde a seleção do cliente até a confirmação do pedido.



O fluxo demonstra a separação de responsabilidades: o Sistema orquestra as operações, delegando persistência aos Repositórios e mantendo a lógica de negócio (calcularTotal) em si mesmo.

2.4 Princípios e Propriedades de Projeto

Esta seção descreve como cada princípio de projeto foi aplicado no Sistema de Cafeteria.

Integridade Conceitual

O sistema mantém uma visão unificada de seu propósito: gerenciar operações de uma cafeteria de forma simples e organizada. Todas as decisões de projeto — nomenclatura de classes, métodos e fluxos — seguem a linguagem ubíqua do domínio (pedido, produto, cliente, atendente), garantindo coerência entre o código e o modelo de negócio.

Coesão

Cada classe possui uma única responsabilidade bem definida. A classe Produto gerencia exclusivamente dados de produtos. A classe Pedido controla o ciclo de vida de um pedido. A classe ItemPedido encapsula a relação entre produto e pedido. Essa separação facilita manutenção e compreensão do código.

Baixo Acoplamento

As classes de domínio se comunicam por meio de interfaces (ProdutoRepository, ClienteRepository, PedidoRepository). Assim, mudanças na implementação da persistência (ex.: trocar banco de dados) não afetam as classes de negócio. O acoplamento é mantido apenas onde necessário, como na relação entre Pedido e ItemPedido.

Ocultação de Informação

Os atributos das classes são privados ou protegidos, expostos apenas por meio de métodos públicos com semântica clara (getters, setters e métodos de negócio). Detalhes de implementação — como a fórmula de cálculo do total ou a lógica de persistência — não são visíveis ao código cliente.

Princípios SOLID

Princípio	Aplicação no Sistema
S — Single Responsibility	Cada classe tem uma única responsabilidade (Produto, Cliente, Pedido, repositórios separados).
O — Open/Closed	Novas funcionalidades (ex.: novo tipo de relatório) podem ser adicionadas sem modificar as classes existentes.
L — Liskov Substitution	As implementações concretas dos repositórios substituem as interfaces sem alterar o comportamento esperado.

Princípio	Aplicação no Sistema
I — Interface Segregation	Interfaces específicas por entidade (ProdutoRepository, ClienteRepository, PedidoRepository) evitam interfaces grandes e desnecessárias.
D — Dependency Inversion	As classes de domínio dependem de abstrações (interfaces), não de implementações concretas de persistência.

Preferência por Composição em vez de Herança

O sistema utiliza composição na relação entre Pedido e ItemPedido: um Pedido é composto por itens, sem necessidade de herança. Isso torna o modelo mais flexível e evita o problema de hierarquias de herança rígidas. A reutilização de comportamento é feita por delegação, não por extensão.

Lei de Demeter

As classes interagem apenas com seus colaboradores diretos. Por exemplo, o sistema não acessa atributos internos de ItemPedido a partir do Pedido encadeando múltiplas chamadas; em vez disso, ItemPedido expõe o método calcularSubtotal(), e Pedido chama diretamente esse método. Isso reduz o acoplamento implícito e torna o código mais robusto a mudanças internas.

2.5 Testes de Software

Os testes foram implementados utilizando JUnit 5 e Mockito. O objetivo foi validar as funcionalidades isoladas de cada serviço, simulando as dependências externas (repositórios) por meio de mocks.

Testes Unitários — ProdutoService

Os testes unitários verificam o comportamento isolado da classe ProdutoService, cobrindo os cenários de sucesso e de falha no cadastro de produtos.

```
// ProdutoServiceTest.java
@ExtendWith(MockitoExtension.class)
class ProdutoServiceTest {

    @Mock
    private ProdutoRepository produtoRepository;

    @InjectMocks
    private ProdutoService produtoService;

    @Test
    void deveCadastrarProdutoComSucesso() {
        Produto produto = new Produto("Café Espresso", 4.50, true);
        when(produtoRepository.salvar(produto)).thenReturn(produto);

        Produto resultado = produtoService.cadastrar(produto);

        assertNotNull(resultado);
        assertEquals("Café Espresso", resultado.getNome());
        verify(produtoRepository, times(1)).salvar(produto);
    }

    @Test
    void naoDeveCadastrarProdutoComPrecoNegativo() {
        Produto produto = new Produto("Café", -1.0, true);

        assertThrows(IllegalArgumentException.class,
            () -> produtoService.cadastrar(produto));
    }
}
```

Casos de teste cobertos:

- Cadastro bem-sucedido de produto com dados válidos.
- Rejeição de produto com preço negativo (validação de negócio).
- Verificação de que o repositório é chamado exatamente uma vez durante o cadastro.

Testes Unitários — PedidoService com Mocks

Os testes do PedidoService utilizam mocks do PedidoRepository e ClienteRepository, simulando a camada de persistência sem necessidade de banco de dados real.

```
// PedidoServiceTest.java
@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {

    @Mock
    private PedidoRepository pedidoRepository;

    @Mock
    private ClienteRepository clienteRepository;

    @InjectMocks
    private PedidoService pedidoService;

    @Test
    void deveCalcularTotalCorretamente() {
        Produto cafe = new Produto("Café", 4.50, true);
        Produto bolo = new Produto("Bolo", 8.00, true);
        Pedido pedido = new Pedido();
        pedido.adicionarItem(new ItemPedido(cafe, 2));
        pedido.adicionarItem(new ItemPedido(bolo, 1));

        double total = pedidoService.calcularTotal(pedido);

        assertEquals(17.00, total, 0.001);
    }

    @Test
    void deveAtualizarStatusDoPedido() {
        Pedido pedido = new Pedido();
        pedido.setId(1);
        when(pedidoRepository.buscarPorId(1)).thenReturn(pedido);

        pedidoService.atualizarStatus(1, StatusPedido.EM_PREPARO);

        verify(pedidoRepository).salvar(pedido);
        assertEquals(StatusPedido.EM_PREPARO, pedido.getStatus());
    }
}
```

Casos de teste cobertos:

- Cálculo correto do valor total do pedido com múltiplos itens.
- Atualização de status do pedido com verificação de persistência.
- Isolamento da lógica de negócio da camada de dados via Mockito.

Cobertura e Estratégia

Classe Testada	Tipo de Teste	Mocks Utilizados	Casos Cobertos
ProdutoService	Unitário	ProdutoRepository	Cadastro válido, preço negativo, chamada ao repo

Classe Testada	Tipo de Teste	Mocks Utilizados	Casos Cobertos
PedidoService	Unitário	PedidoRepository, ClienteRepository	Cálculo total, atualização de status
ClienteService	Unitário	ClienteRepository	Cadastro de cliente, busca por ID
ItemPedido	Unitário	Nenhum	calcularSubtotal() com diferentes quantidades

O uso de mocks garantiu que os testes sejam rápidos, determinísticos e independentes de infraestrutura externa, em conformidade com os princípios de testes unitários eficazes.